

Teaching a machine to call a pit stop

A reinforcement learning project that failed at what it set out to do, and found four better things on the way.

The problem

In a twenty-four-hour race, the fastest car does not always win. The car that stops at the right moments does.

Endurance racing is decided in the pit lane. A stop costs a minute and a half and you will take twenty-five of them; whether each one costs you a place depends on what the rest of the field is doing, on whether a caution period is about to bunch everyone up, and on whether the pit lane is even open to your class when you arrive. Get the timing right and you gain a lap on the field for free. Get it wrong and you spend the next four hours recovering.

That makes it a decision problem with a shape reinforcement learning ought to like: a long sequence of small choices, a delayed and noisy payoff, and a strategic structure that is genuinely hard for a person to reason about at the scale of a whole race.

I wanted to know whether a learned policy could beat human-style pit strategy. It could not, and the reason turned out to be far more interesting than the question.

What this project actually produced:

a race simulator calibrated from 1.66 million laps of real timing data, in which a full 24-hour race with 62 cars runs in about half a second

both rulebooks — WEC and IMSA — where they genuinely differ, which turns out to be the difference between a strategy working and not working

six pit strategies scored against each other over two hundred paired races apiece, with confidence intervals

four measured ways this simulator misleads a learner, each measured directly on the engine rather than inferred from how a policy scored

an evaluation protocol that can tell a policy which has learned something from one which has learned to do nothing — because mine had learned to do nothing, and nothing I had built could see it

The data, and the first thing it did to me

The source is a single file: 1,658,803 laps, 582 MB, four championships, six seasons, 1,013 sessions. Every lap of every car, with sector times, tyre age, flag state and pit time.

My first calibration produced a race that ran for **216 hours** with green stints of **199 laps**, and tyres that got *faster* the longer they were used.

None of those numbers is a bug in the arithmetic. They are all the same mistake: the query scoping the data to "the Daytona 24" was matching on the event name, and the file holds nine runnings of it. I was calibrating a race that was nine Daytonas stacked on top of each other. The lap counter ran straight through them, so a "stint" spanned editions; the duration was nine races long; and pooling nine grids of different cars destroyed the relationship between tyre age and lap time that the degradation fit depends on.

The fix was one column — the file has a `session_id`, and one race is one session. But finding it meant profiling the file rather than trusting it, and that turned up three more things of the same kind:

Car #7 and car #007 are two different cars. Both are Hypercars at Le Mans. Read as integers they become one car with twice the laps and twice the stops, and one car vanishes from the grid.

The stint counter does not count stints. It steps on driver changes, which happen about every third stop. Reading it as a fuel stint reported 58-lap green stints at Daytona, in a race the winner ran in stints of 23.

The chequered lap is not a caution. The flag column has five values and my first pass counted everything that was not green as a caution, which quietly added one caution period per car per race.

Each of these produced output that looked entirely plausible. None of them raised an error. That is the whole lesson of the data stage, and it is why every later stage in this project carries a check that is designed to *fail* on a specific fault rather than merely to pass.

What could not be fixed. The pit time column carries hour-long repairs alongside ordinary service, so every pit figure is a trimmed median rather than a mean. And at Daytona the front-running classes change tyres at every stop, which makes tyre age and fuel load the same number — no fit can separate them, so the calibration reports the degradation slope as *unidentified* rather than returning a figure it cannot support. At Le Mans the fitted slope for Hypercar and LMP2 comes out **negative**, which cannot be tyre wear. It is fuel burning off and the track rubbering in, and the data cannot pull them apart. The simulator applies it as though it were tyre wear, and the app says so on every screen.

A simulator you can argue with

The engine runs on five dials per class per series: pace and degradation, a caution rate and duration, a stint length held as fuel, a pit cost, and a traffic density. Everything else is derived.

The split that matters is not between dials — it is between **what the timing data can measure and what it cannot**. Lap timing records that a lap was slow; it never records how close the car ahead was. It records how long a stop took; it never records what happened during it. So a set of quantities — how much a caution slows the field, how tightly cars bunch behind a safety car, how a stop divides between driving down the lane, changing tyres and putting fuel in — are **assumed**, marked as such everywhere they appear, and exposed as sliders in the app.

That is not a disclaimer. It is the design. The honest response to a number somebody chose is to move it and see whether the conclusion survives, so every assumed dial is swept rather than trusted.

One statement sits above all of them. These dials come from *one running of one race per series* — Daytona 2026 and Le Mans 2026. Between two adjacent Daytonas the share of the race spent under caution moves by a factor of 2.8. So the width of each slider is a better guide to the uncertainty than its starting value is.

Where the two rulebooks stop being one model

This is the part I enjoyed most, and it is the part that decides the results.

When a caution comes out, the pit lane closes. **IMSA** reopens it in class order — prototypes first, then GTs, then everyone (art. 46.3.1) — and has a Short Full-Course-Yellow rule under which a caution near the start, near a restart, or near the finish never opens the lane at all (art. 46.3.3). **WEC** releases the whole field together once the closure expires (art. 14.6.5).

IMSA also permits four people over the wall including the refueller, so tyres come off while fuel goes in and a stop costs the *longer* of the two jobs. WEC forbids tools during refuelling, so the jobs are sequential and a stop costs their *sum*.

Both series wave lapped cars back onto the lead lap under caution, using identical eligibility rules — but IMSA runs it twice per caution and WEC once.

None of that is decoration. A strategy built around gambling on caution stops gains a place in 58% of Daytona races and is worth much less at Le Mans, entirely because of who gets released from a closed pit lane and when.

How anything here gets compared

Racing is noisy, and a strategy that looks good over five races is usually a strategy that got five good races. **Every strategy is scored on the same race, run twice.** Once with the strategy driving the focal car, once with a plain fuel-window plan driving it. The difference between those two runs is the strategy's effect on *that* race, with the caution timeline, the traffic and the other sixty cars held identical. Two hundred such pairs per strategy per series, with bootstrap intervals.

That only works if the noise is a property of the race rather than of the strategy — and at first it was not. Under a single shared random generator, a car that pitted earlier drew different lap-time noise for the rest of the race, so a comparison measured the strategy *and* a reshuffle. Now every random number is drawn from the seed before the race starts, and each car's noise lives in an indexed stream, so a car's third pit stop gets its third noise draw whenever it takes it. Two strategies stopping at completely different moments see identical noise on identical laps.

The check on all of this is simple enough to print in the results: **the baseline scored against itself must move nothing at all, on every seed.** If that row is not exactly zero, the apparatus is wrong and no number beside it means anything.

It earned its place. An early comparison found a two-place gap between two strategies — in a statistic whose spread from race to race ran to three places. Two hundred races reversed its direction.

What the human strategies showed

Six strategies, none of which has a tunable number in it: each derives its thresholds from the dials, so none can be quietly fitted to the races it is scored on.

At Daytona, gambling on cautions works. Stopping when a caution is called — when the field is slow and the lane transit is cheap — gains a place in **58% of 200 races**, with a median gain of one place, and it holds up on fifty races the strategy was never selected on. It is the strongest single result the project has.

At Le Mans it mostly does not, and for a rulebook reason rather than a racing one: WEC releases everyone from the closed lane together, so far fewer caution stops are reachable and the gamble has less to win. The strategy that does well there is the one that plans its final fill to reach the flag exactly — it gains in 45% of races and *loses* in 1%, which is a very different shape of result from the gambler's.

And one negative result worth reporting. I built a two-stage reference to score strategies against what was actually achievable rather than only against each other: one arm knowing the caution timeline in advance, one arm not, with the gap between them measuring the value of foreknowledge. On real dials it is degenerate. The clairvoyant arm extracts *zero* foreknowledge on every seed and is beaten by its own control. Rather than show a number that does not mean what it appears to, there is no benchmark row anywhere in the app.

The agent, and why there is no agent result

The policy sees ten numbers at each crossing of the line — race progress, fuel, tyre age, the gaps ahead and behind, the flag, stint length, laps down, whether the lane is open, and its class position — and chooses among five actions: stay out, or one of four combinations of fuel and tyres. It is trained with

masked PPO, and it is scored by being inserted into the roster as a seventh member and run through exactly the same function the six others go through. There is deliberately no agent-specific evaluation code: a second path that can differ from the roster's produces plausible numbers rather than errors.

Every agent figure this project produced came from a single training run.

The human strategies are deterministic functions of the race, so their two-hundred-race interval is the whole of their uncertainty. An agent's row carries that interval *plus* the spread across training seeds — and nothing had ever measured the second. So I trained five, changing nothing but the seed.

In WEC there is no result to report. Across five seeds the headline statistic ranged from 0.000 to 0.450 — one run gained a place in 45% of races and three others gained one in *none*. That spread is fifteen times what two hundred paired races can resolve. The five policies were not variations on one behaviour: their median stop counts ran 38, 65, 72, 174, 194 in a race where a sensible number is about forty.

In IMSA it was worse, because it looked stable. All five seeds came out identical to three decimal places on every column. A policy that had learned something would vary. So I built `never_pit` — a strategy that always stays out and lets the engine's out-of-fuel rule supply every stop — and scored it against the roster. It scored the same as the trained policy. On the fifty held-out races the two agree to three decimal places on every column.

All five training runs had converged on **asking for nothing at all**. "Always stay out" is one deterministic behaviour, so five different sets of weights produce five identical races. What looked like a stable result was a policy that had learned to take no decisions.

`never_pit` is now the sixth member of the roster, and it is the true null for a learner: a policy that cannot beat *taking no decisions* has learned nothing. It costs one small class and it would have caught this four retracts earlier. Nothing I had built before could have seen it.

So both headline agent figures are withdrawn, and no number is reported for the policy anywhere in the app. It can still be watched taking decisions lap by lap, with the reason it carries no number written beside it.

Four ways the simulator misled the learner

This is what survives, and it is better than what was withdrawn. Each was measured on the engine directly, so none depends on any training run.

1. A stop was priced below its own pit lane. The cheapest caution stop the engine could produce cost 13.47 s against a 22.45 s lane transit — the time to drive the length of the pit lane with nobody touching the car. A caution discount meant to represent "time in the box matters less when the field is slow" was being applied to the drive down the lane as well. The policy found it and stopped 164 times in a 189-lap race. That is not a bug in the policy; it is the policy being right about the model.

2. Caution compression severs laps from the score. A car that takes forty extra stops at Daytona spends 1,072 seconds more in the pits and loses **zero laps**. Behind a safety car the field bunches up, so time lost in the pits is handed back on the road. The refund tracks the caution rate directly: 70% at Daytona's rate, 35% at a third of it, 8% at a near-green race. I had been rewarding laps completed, on the reasoning that class position derives from laps and then time. At Daytona's caution rate it does not.

3. The credit horizon is shorter than the payback window. A forced extra caution stop, in an otherwise identical race:

20 laps later at the flag

Daytona **+0.89 places** -0.67 places

Le Mans -0.45 places -0.82 places

At Daytona the sign flips. The learning algorithm's credit assignment puts essentially all of the weight inside about twenty steps, so the agent was told the truth about the next twenty laps and the opposite about the race. At Le Mans the near-term value is already negative and there is no trap at all — which is the cleanest explanation I have for why the two series behaved so differently throughout.

4. The reward was defined on something the agent could not see. Once the reward became class position, the observation row meant to carry position — laps behind the class leader — turned out to correlate **0.091** with actual class position, because it is clipped at three laps and a front-runner sits at zero all race. The value function was estimating "how many places will I gain from here" without being told where *here* was. The fix was a tenth observation row, correlating 1.000 with position, which is the least surprising sentence in this write-up and took four retrains to reach.

They share one root. Caution compression makes stopping locally cheap, and a cost, a proxy or a horizon can each fail to see that differently. That is the finding this project is actually about.

The dial that cannot be measured

One assumed number decides a great deal: what share of a pit stop is driving down the lane, as opposed to being serviced. It is what makes a splash-and-dash cheaper than a full service, and it is the number the policy exploited in trap one. I set it to 0.25 and wanted to know what it really is.

It cannot be known from this data, and I can show that in three ways.

The obvious estimator is a low quantile of real pit times — the cheapest stops are presumably the ones with least service in them. Against a synthetic race with a planted answer of 0.407, that estimator returns 0.548. It is biased high by about a seventh, in the direction you would expect: the cheapest stops in any real sample still have *some* service in them.

It is also not stable. Across the seven classes of these two races the same statistic gives 0.50, 0.50, 0.52, 0.73, 0.88, 0.90, 0.93. A measurement that ranges from half a stop to nearly all of one depending on which class you pick is not measuring a property of a pit lane.

So I tried to separate the fixed part from the variable part properly, by regressing pit time on how much fuel went in — proxied by how far the car got before it came back. That has no leverage at all, and the reason is the sport rather than the method: **endurance cars fill to the brim.** Between 62% and 90% of stops in these two races are followed by a near-full tank, and the median stop is a brim-full tank in all seven classes. Separating what a stop costs to *enter* from what it costs to *fill* requires observing stops that took different amounts of fuel, and this sport barely produces any.

Two intermediate results came out of that work and both had to be withdrawn. The first reported that 95% of a stop was fixed cost — an artefact, because a regressor carrying no signal produces a flat line, and a flat line hands its entire budget to the intercept, which is then just the average stop. The second converted the failed fit into a lower bound of 84% to 98% — also withdrawn, because it assumed a short stint means a small fill, and a stint also ends early for a caution or for the flag, with nothing in the data to say which.

The rule I took from that: **a refusal that offers a number anyway is a refusal nobody heeds**. The dial stays at its assumed value, is marked as having no measured counterpart, and is swept — which is exactly what it was put in the assumed list for.

What I would do next

Train for races, not laps. The budget is counted in decision steps, and Daytona costs 742 of them per race against Le Mans's 385 — so the same budget buys 674 Daytonas and 1,299 Le Mans. That asymmetry ran through the whole project untested.

Report an agent as a distribution. Two hundred paired races exist because one race means nothing; a single training run is the same mistake one level up. Five seeds should be the floor, and the stop count should be reported beside the score, because two runs can score alike for entirely different reasons.

Rank the four traps. Each is measured, but which dominates cannot be settled without training runs that resolve, and at half a million steps they do not.

Change the shape of a pit stop. A full service is anchored at the measured mean and split into shares, so raising one share lowers another and a full stop always costs the same. That is the right shape for asking what a *partial* stop saves and the wrong shape for a real pit lane, where entry, the speed limit and exit are most of the cost whatever you stopped for.

What this does not claim

The agent did not beat, match or lose to the human strategies — there is no agent result to compare with. The four traps are not ranked against each other. The dials describe one running of one event per series and are not an average of anything.

And the simulator is a model. It reproduces lap times, pit costs, caution behaviour, traffic and the two rulebooks. It does not reproduce weather, contact, damage, penalties, or a driver having a bad night — all of which decide real endurance races and none of which is in the data I had.

Everything above is reproducible from this repository. Every figure quoted traces to a file on disk, every artefact carries a hash of the dials and the rulebook logic it was built against, and anything that does not match is refused rather than quietly used. `python scripts/check_artefacts.py` reports whether the tree hangs together; `pytest tests/` runs 266 tests, each stage's verification gate among them.